



Topic 9: Error Detection and Correction

CSE 4405: Data and Telecommunications

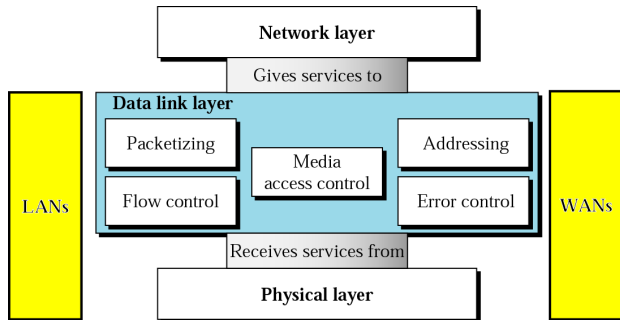
Md. Tanvir Hossain Saikat, Jr. Lecturer

Islamic University of Technology (IUT)

Where Error Control Fits

- Transmission media and channels can corrupt bits.
- The data-link layer is responsible for local frame reliability.
- Some transport-layer protocols also use error checking end to end.
- This chapter explains the redundancy used to detect or correct errors.

Error control is one of the key services associated with the data-link layer.



Learning Objectives

1. Explain single-bit and burst errors in transmitted data.
2. Distinguish error detection, error correction, forward error correction, and retransmission.
3. Explain redundancy, modulo-2 arithmetic, and block-code terminology.
4. Use Hamming distance and minimum distance to reason about error-control capability.
5. Describe linear block codes, simple parity, two-dimensional parity, and Hamming codes.
6. Explain cyclic codes, CRC generation/checking, generator polynomials, and CRC limitations.
7. Compute checksum examples using one's-complement arithmetic.

Source focus: Forouzan, *Data Communications and Networking*, Chapter 10.

10.1 Introduction

- Types of errors
- Redundancy
- Detection versus correction
- Forward error correction versus retransmission
- Coding and modular arithmetic

Core techniques

- 10.2 Block coding
- 10.3 Linear block codes
- 10.4 Cyclic codes and CRC
- 10.5 Checksum

Unifying idea

Every method in this chapter adds carefully chosen redundant information so that the receiver can notice, reject, or sometimes repair corrupted data.

10.1 Introduction

Central problem

A receiver sees only the bits that arrive, not the original bits that were sent. Error-control coding gives the receiver extra structure to detect or correct corruption.

This section introduces

- the two common error patterns: single-bit and burst errors;
- why redundancy is necessary;
- the difference between detecting and correcting errors;
- the tradeoff between forward correction and retransmission;
- modulo-2 arithmetic, the arithmetic used by many codes.

Types of Errors

Single-bit error

Only one bit in the data unit changes value. These are easier to reason about and often used to introduce error-control ideas.

Burst error

Two or more bits in the data unit are affected within a span. Not every bit inside the span must be corrupted.

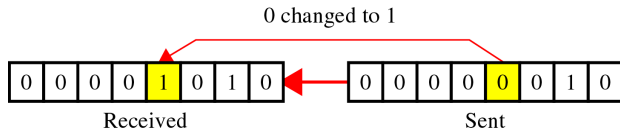
Lecture point

A code that handles isolated bit flips may still perform poorly when a channel produces burst errors. Error pattern matters.

Single-Bit Error

Figure: Single-bit corruption

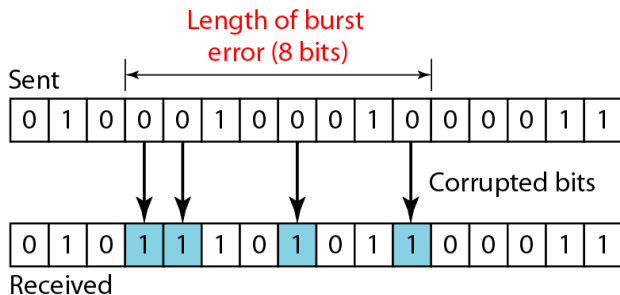
- The sent and received data units differ in exactly one position.
- Here, a transmitted 0 is received as 1.
- Detection is usually easier than correction because correction must identify the bit location.



Burst Error

Figure: Burst-error length

- The burst length is measured from the first corrupted bit to the last corrupted bit.
- Some bits inside the span may remain unchanged.
- Burst errors are common when noise or fading affects the channel for an interval of time.



Redundancy

Definition

Redundancy means appending extra bits to a data unit instead of retransmitting the entire data stream.

- The added bits are shorter than the original data unit.
- The sender computes the extra bits from the data.
- The receiver checks the relationship between data bits and redundant bits.
- After the check, the redundancy is discarded or used for correction.

To detect or correct errors, a system must send extra redundant bits with the data.

Error Detection Versus Error Correction

Error detection

The receiver decides whether an error has occurred. It does not need to know exactly which bit is wrong.

Error correction

The receiver must infer the original codeword, which usually means identifying the number and location of corrupted bits.

Why correction is harder

Detection asks, “Is this valid?” Correction asks, “Which valid message was most likely sent?”

Forward Error Correction Versus Retransmission

Forward error correction

The receiver uses redundant bits to guess or reconstruct the original message without asking the sender to repeat it.

Retransmission

The receiver detects an error and asks the sender to send the message again.

- Forward correction adds more overhead before errors occur.
- Retransmission depends on feedback and adds delay when errors occur.
- Practical systems choose based on delay, channel quality, and complexity.

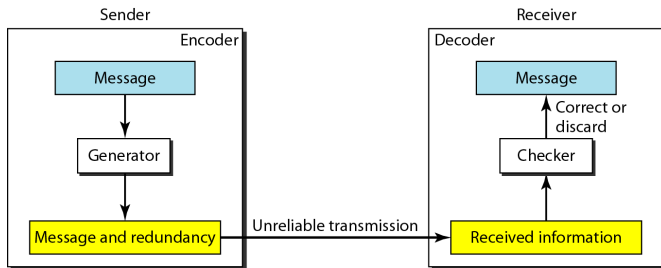
- Redundancy is created through coding schemes.
- This chapter focuses on **block coding**;
- The sender creates a relationship between data bits and redundant bits.
- The receiver checks that relationship to detect or correct errors.

Coding family	Basic idea
Block coding	Protect fixed-size datawords by adding check bits.
Convolutional coding	Protect a bit stream using encoder memory.

Coding Structure

Figure: Encoder and decoder

- The encoder adds redundancy before transmission.
- The channel may corrupt the protected message.
- The decoder checks the received information and either accepts, corrects, or discards it.



Modular Arithmetic

Modulo- N arithmetic

Only the integers from 0 to $N - 1$ are used. The value N is the modulus.

- Values wrap around when they reach the modulus.
- Many error-control codes use modulo-2 arithmetic.
- In modulo-2 arithmetic, the only symbols are 0 and 1.

Modulo-2 view

Modulo-2 addition and subtraction are both equivalent to XOR. This is why XOR appears throughout block codes and CRC.

Modulo-2 Arithmetic

- Addition:

$$0 + 0 = 0 \quad 0 + 1 = 1$$

$$1 + 0 = 1 \quad 1 + 1 = 0$$

- Subtraction gives the same results in modulo-2 arithmetic.
- XORing two words produces 1 exactly where their bits differ.

Figure: XORing bits and bit patterns

$0 \oplus 0 = 0$	$1 \oplus 1 = 0$
------------------	------------------

a. Two bits are the same, the result is 0.

$0 \oplus 1 = 1$	$1 \oplus 0 = 1$
------------------	------------------

b. Two bits are different, the result is 1.

	1	0	1	1	0
$\oplus$	1	1	1	0	0
	0	1	0	1	0

c. Result of XORing two patterns

10.2 Block Coding

Definition

A message is divided into k -bit datawords. The encoder adds r redundant bits, producing n -bit codewords where $n = k + r$.

Topics in this section

- Error detection
- Error correction
- Hamming distance
- Minimum Hamming distance

A block code maps 2^k possible datawords into a selected set of valid n -bit codewords.

Datawords and Codewords

Figure: Block-code expansion

- Each dataword has k bits.
- Each codeword has n bits.
- Only 2^k of the 2^n possible n -bit patterns are valid codewords.



2^k Datawords, each of k bits



2^n Codewords, each of n bits (only 2^k of them are valid)

Example 10.1: 4B/5B as Block Coding

Known example from digital transmission

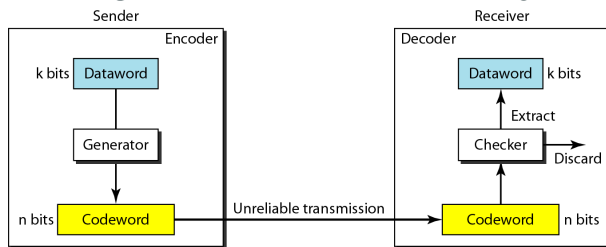
In 4B/5B coding, $k = 4$ and $n = 5$. There are $2^4 = 16$ datawords and $2^5 = 32$ possible 5-bit words.

- Sixteen codewords are used for message transfer.
- The remaining 5-bit patterns are reserved for other purposes or left unused.
- This illustrates the block-code idea: not every possible n -bit pattern is valid.

Block Coding for Error Detection

- Detection is possible when the receiver knows the valid codewords.
- If a sent codeword changes into an invalid word, the receiver rejects it.
- If errors change one valid codeword into another valid codeword, the error is not detected.

Figure: Error detection in block coding



Example 10.2: A Code for Detection

Dataword	Codeword
00	000
01	011
10	101
11	110

- Here $k = 2$ and $n = 3$.
- Suppose the sender encodes dataword 01 as codeword 011.
- If the receiver gets 011, the received word is valid and dataword 01 is accepted.
- If the receiver gets 111, the word is invalid and is discarded.

Example 10.2: Undetected Error

Case

The sender transmits 011, but two bit errors produce 000 at the receiver.

- 000 is a valid codeword.
- The receiver extracts dataword 00.
- The error is not detected because the corrupted word is still valid.

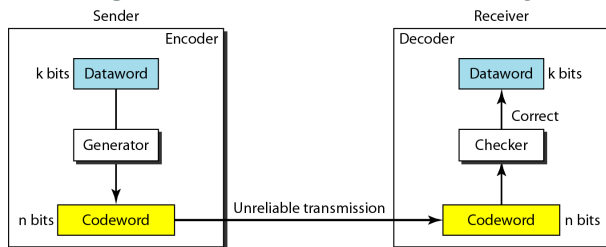
Lesson

Error detection depends on how far apart the valid codewords are. Valid codewords that are too close can be confused by multiple bit errors.

Block Coding for Error Correction

- In correction, the receiver must infer the original codeword.
- More redundancy is needed than for detection alone.
- The decoder chooses the valid codeword that best matches the received pattern.

Figure: Error correction in block coding



Example 10.3: A Code for Correction

Dataword	Codeword
00	00000
01	01011
10	10101
11	11110

- Three redundant bits are added to each 2-bit dataword.
- Dataword 01 becomes codeword 01011.
- Suppose transmission corrupts it into 01001.
- The receiver first notes that 01001 is not a valid codeword.

Example 10.3: Nearest Valid Codeword

- Compare received word 01001 with each valid codeword.
- 01001 differs from 00000 in two bit positions.
- It also cannot be the third or fourth codeword by the same distance reasoning.
- It differs from 01011 in only one bit position.
- Assuming only one bit was corrupted, the receiver replaces 01001 with 01011 and recovers dataword 01.

Correction rule

Correction is possible when each likely received pattern is closer to one valid codeword than to all others.

Hamming Distance

Definition

The Hamming distance between two equal-length words is the number of bit positions in which they differ. It is written as $d(x, y)$.

- XOR the two words.
- Count the number of 1s in the XOR result.
- That count is the Hamming distance.

Hamming distance is the basic measurement of how separated two codewords are.

Example 10.4: Computing Hamming Distance

$$d(000, 011) = 2$$

$$000 \oplus 011 = 011$$

The result contains two 1s.

$$d(10101, 11110) = 3$$

$$10101 \oplus 11110 = 01011$$

The result contains three 1s.

A distance of 3 means three bit positions are different between the two words.

Minimum Hamming Distance

Definition

The minimum Hamming distance, $d_{\min}$, is the smallest Hamming distance among all pairs of valid codewords in a coding scheme.

- It measures the worst-case separation between valid codewords.
- Larger $d_{\min}$ generally gives stronger detection or correction capability.
- $d_{\min}$ is computed only among valid codewords, not all possible bit patterns.

Examples 10.5 and 10.6: Minimum Distance

Table 10.1 code

Valid codewords: 000, 011, 101, 110.

All nonzero pair distances are 2, so:

$$d_{\min} = 2$$

Table 10.2 code

Valid codewords: 00000, 01011, 10101, 11110.

The smallest pair distance is 3, so:

$$d_{\min} = 3$$

Meaning

The second code spaces its valid codewords farther apart, which is why it can support correction in Example 10.3.

Code Parameters and Errors

- A block code is commonly written as $C(n, k)$.
- n is the number of bits in each codeword.
- k is the number of data bits in each dataword.
- $d_{\min}$ is often listed separately because it describes error-control strength.

Original examples

The first code is $C(3, 2)$ with $d_{\min} = 2$. The second code is $C(5, 2)$ with $d_{\min} = 3$. If 00000 is sent and 01101 is received, three bits are in error, so the Hamming distance is 3.

Minimum Distance for Error Detection

Detection guarantee

To guarantee detection of up to s errors in all cases, a block code must satisfy

$$d_{\min} = s + 1.$$

- With s or fewer errors, a valid codeword cannot be changed into another valid codeword.
- The received word therefore lands outside the valid-codeword set.
- The receiver can reject it as corrupted.

Example 10.7: $d_{\min} = 2$

Single error is detected

If 101 is sent and one error occurs:

$$101 \rightarrow 100$$

100 is not a valid codeword.

Two errors may be missed

If 101 is sent and two errors occur:

$$101 \rightarrow 000$$

000 is valid, so the receiver is fooled.

The Table 10.1 code guarantees only single-error detection.

Example 10.8: $d_{\min} = 3$

- The second block code has $d_{\min} = 3$.
- It can detect up to two errors.
- Any one- or two-bit corruption takes a valid codeword outside the valid-codeword set.
- Some three-error patterns can still change one valid codeword into another.

Lesson

A higher minimum distance improves detection, but no finite block code catches every possible corruption pattern.

10.3 Linear Block Codes

Definition

A linear block code is a block code in which XORing any two valid codewords produces another valid codeword.

- Almost all practical block codes belong to this family.
- Linearity makes analysis and implementation simpler.
- This section covers minimum distance for linear codes and three examples: simple parity, two-dimensional parity, and Hamming codes.

Example 10.10: Checking Linearity

- The Table 10.1 code is linear: XORing any two codewords gives another valid codeword.
- Example:

$$011 \oplus 101 = 110$$

- The Table 10.2 code is also linear.
- Its valid codewords can be created by XORing other valid codewords in the set.

Why it matters

Linearity lets us study the code by studying XOR sums and the weights of codewords.

Minimum Distance for Linear Block Codes

Simplification

For a linear block code, $d_{\min}$ is the number of 1s in the nonzero valid codeword with the smallest number of 1s.

- The number of 1s in a word is its Hamming weight.
- Instead of comparing every pair, inspect nonzero valid codewords.
- The smallest nonzero weight gives $d_{\min}$.

Example 10.11: Linear-Code Distance

Table 10.1

Nonzero codewords: 011, 101, 110.

Their weights are 2, 2, and 2.

$$d_{\min} = 2$$

Table 10.2

Nonzero codewords: 01011, 10101, 11110.

Their weights are 3, 3, and 4.

$$d_{\min} = 3$$

Simple Parity-Check Code

- A k -bit dataword becomes an n -bit codeword where $n = k + 1$.
- The extra bit is the parity bit.
- The parity bit is chosen so the total number of 1s is even or odd.
- The code can detect a single-bit error, but it cannot correct an error.

A simple parity-check code has $n = k + 1$ and $d_{\min} = 2$.

Table 10.3: Simple Parity-Check Code

Figure: Simple parity-check code $C(5, 4)$

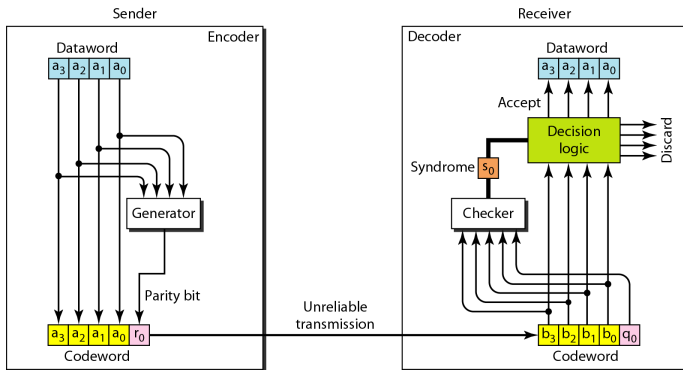
- Four data bits are extended by one parity bit.
- The table lists all 16 datawords and their 5-bit codewords.
- Each codeword has even parity.

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Simple Parity Encoder and Decoder

Figure: Encoder and decoder

- The generator computes the parity bit.
- The checker recomputes parity at the receiver.
- Syndrome 0 means the parity check passes; syndrome 1 means an error is detected.



Example 10.12: Simple Parity Cases I

Setup

The sender sends dataword 1011. With even parity, the codeword is 10111.

1. No error: received 10111, syndrome 0, dataword 1011 is accepted.
2. One data bit changes: received 10011, syndrome 1, no dataword is created.
3. The parity bit changes: received 10110, syndrome 1, no dataword is created.

Example 10.12: Simple Parity Cases II

4. Two bits change, r_0 and a_3 : received 00110, syndrome 0, but the wrong dataword 0011 is created.
5. Three bits change, a_3 , a_2 , and a_1 : received 01011, syndrome 1, so no dataword is created.

Conclusion

Simple parity detects all odd numbers of bit errors, but even-numbered error patterns can pass undetected.

A simple parity-check code can detect an odd number of errors.

- It cannot identify which bit is wrong.
- It cannot detect all even-numbered error patterns.
- It is useful because it is very cheap, not because it is strong.

Two-Dimensional Parity Check

- Data is arranged in a table of rows and columns.
- One parity bit is computed for each row.
- One parity bit is computed for each column.
- The receiver recomputes row and column syndromes.
- Two-dimensional parity can detect up to three errors anywhere in the table.

Why it helps

Row and column checks give location information that a single parity bit cannot provide.

Two-Dimensional Parity: Construction

Figure: Row and column parities

- Each row receives a parity bit.
- Each column receives a parity bit.
- The lower-right bit checks the parity of the parity bits.

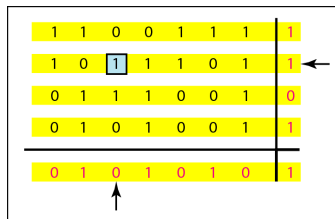
1	1	0	0	1	1	1	1	Row parities
1	0	1	1	1	0	1	1	
0	1	1	1	0	0	1	0	
0	1	0	1	0	0	1	1	
Column parities								1

a. Design of row and column parities

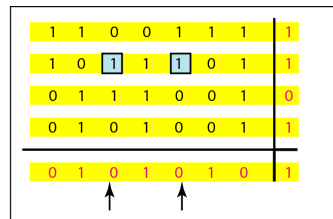
Two-Dimensional Parity: One and Two Errors

Figure: Syndrome patterns

- One bit error affects one row parity and one column parity.
- Two errors can affect multiple parity checks.
- These patterns make the corruption visible to the receiver.



b. One error affects two parities

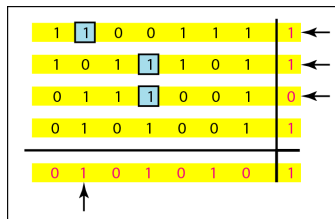


c. Two errors affect two parities

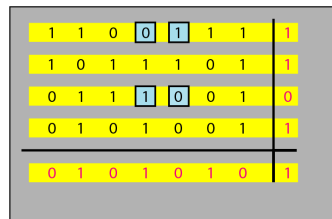
Two-Dimensional Parity: Limits

Figure: Three and four errors

- Three errors are still detected in the shown pattern.
- Some four-error rectangles can leave all parities consistent.
- This illustrates that stronger redundancy still has limits.



d. Three errors affect four parities



e. Four errors cannot be detected

Hamming Codes

- Hamming codes were originally designed with $d_{\min} = 3$.
- They can detect up to two errors or correct one single-bit error.
- Choose an integer $m \geq 3$.
- The code length is $n = 2^m - 1$.
- The data length is $k = n - m$, and the number of check bits is $r = m$.

For the Hamming codes discussed here, $d_{\min} = 3$ and $n = 2^m - 1$.

Table 10.4: Hamming Code $C(7, 4)$

Figure: Codewords for $C(7, 4)$

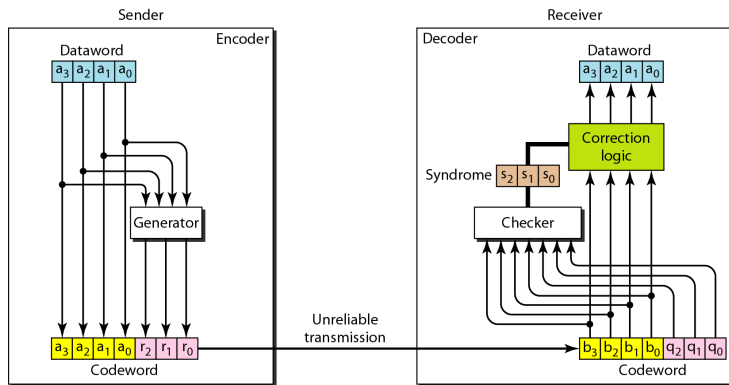
- Four information bits are protected by three parity bits.
- The table lists all 16 valid codewords.
- The highlighted bits are the redundant bits.

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	0000 000	1000	1000 110
0001	0001 101	1001	1001 011
0010	0010 111	1010	1010 001
0011	0011 010	1011	1011 100
0100	0100 011	1100	1100 101
0101	0101 110	1101	1101 000
0110	0110 100	1110	1110 010
0111	0111 001	1111	1111 111

Hamming Encoder and Decoder

Figure: Hamming-code structure

- The generator computes three check bits.
- The checker computes a 3-bit syndrome.
- Correction logic uses the syndrome to decide which bit to flip.



Calculations for a Hamming Code

Generator parity bits

$$r_0 = a_2 + a_1 + a_0 \pmod{2}$$

$$r_1 = a_3 + a_2 + a_1 \pmod{2}$$

$$r_2 = a_1 + a_0 + a_3 \pmod{2}$$

Decoder syndrome bits

$$s_0 = b_2 + b_1 + b_0 + q_0 \pmod{2}$$

$$s_1 = b_3 + b_2 + b_1 + q_1 \pmod{2}$$

$$s_2 = b_1 + b_0 + b_3 + q_2 \pmod{2}$$

All additions are modulo-2 additions, so each equation is implemented with XOR logic.

Syndrome Decision Logic

Figure: Table 10.5

- The syndrome points to the bit suspected of being wrong.
- If syndrome is 011, bit b_2 must be flipped.
- If syndrome is 000, no correction is needed or only a parity bit was affected.

<i>Syndrome</i>	000	001	010	011	100	101	110	111
<i>Error</i>	None	q_0	q_1	b_2	q_2	b_0	b_3	b_1

Example 10.13: Hamming-Code Decoding

Dataword	Codeword sent	Codeword received	Syndrome	Decoder action
0100	0100011	0100011	000	Accepts final dataword 0100.
0111	0111001	0011001	011	Flips b_2 , recovers final dataword 0111.
1101	1101000	0001000	101	Flips b_0 , producing wrong dataword 0000; the code cannot correct two errors.

Limit

A Hamming code with $d_{\min} = 3$ can correct one error, but not every two-error pattern.

Example 10.14: Choosing m , n , and k

Problem

We need a dataword of at least 7 bits. Find values of k and n for a Hamming code that satisfy this requirement.

- Need $k = n - m \geq 7$.
- Since $n = 2^m - 1$, the requirement is $2^m - 1 - m \geq 7$.
- If $m = 3$: $n = 7$, $k = 4$, not enough.
- If $m = 4$: $n = 15$, $k = 11$, sufficient.

The resulting code is $C(15, 11)$.

Burst Error Correction with Hamming Codes

- A Hamming code can correct a single error or detect a double error.
- A burst error may corrupt several adjacent bits in a transmitted data unit.
- Burst correction can be improved by distributing adjacent transmitted bits across several codewords.
- The key is to make one burst appear as at most one error in each separate codeword.

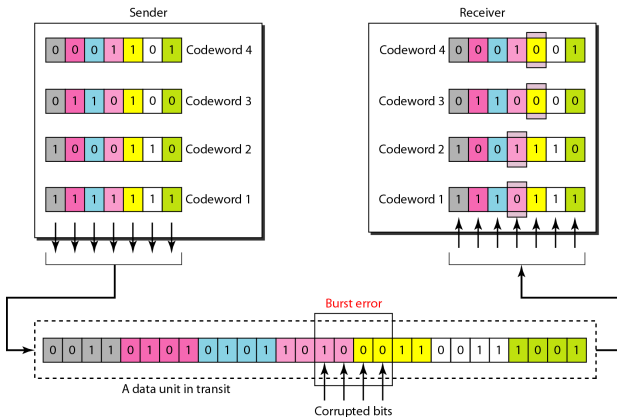
Technique

Interleaving separates neighboring transmitted bits before coding or decoding, so a burst in the channel is spread across codewords.

Burst Error Correction Using Hamming Code

Figure: Interleaving idea

- The sender interleaves bits from multiple codewords.
- The burst damages one position from several codewords.
- The receiver deinterleaves and corrects one error per codeword.



10.4 Cyclic Codes

Definition

Cyclic codes are linear block codes with one extra property: if a codeword is cyclically shifted, the result is also a codeword.

Topics in this section

- Cyclic redundancy check (CRC)
- Hardware implementation
- Polynomials
- Cyclic-code analysis
- Advantages and other cyclic codes

Cyclic Property and CRC

- In a cyclic code, rotating a valid codeword gives another valid codeword.
- Example: if 1011000 is a codeword, then a cyclic left shift gives 0110001.
- The last bit wraps around and becomes the first bit.
- The cyclic redundancy check, or CRC, is widely used in networks such as LANs and WANs.

CRC is a cyclic-code method built around binary division.

Table 10.6: A CRC Code $C(7, 4)$

Figure: CRC codewords

- Four-bit datawords map to seven-bit codewords.
- The extra bits are the CRC remainder.
- This table is another valid block-code mapping.

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

CRC Based on Binary Division

CRC rule

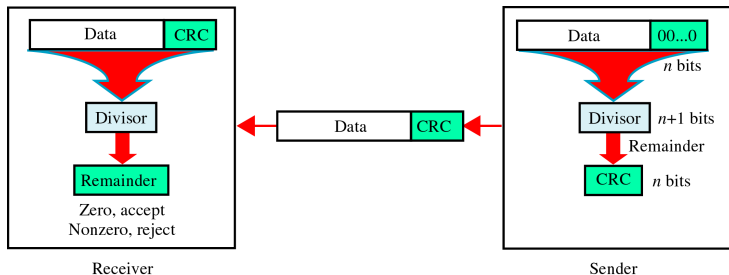
The sender appends a CRC so that the full sequence, data plus CRC, is exactly divisible by the chosen divisor.

- The CRC has exactly one fewer bit than the divisor.
- The sender computes the remainder after dividing an augmented dataword.
- The receiver divides the received codeword by the same divisor.
- Remainder 0 means the frame passes the CRC check; nonzero remainder means it is rejected.

CRC Encoder and Decoder

Figure: CRC generation and checking

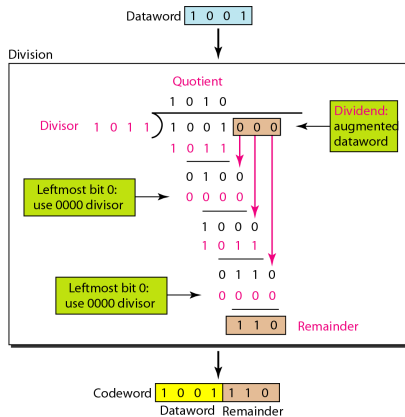
- Sender: divide augmented data by the divisor and append the remainder.
- Receiver: divide the received codeword by the same divisor.
- A zero remainder accepts; a nonzero remainder rejects.
- The same division logic maps directly to shift-register and XOR hardware.



CRC Division at the Encoder

Figure: Division in CRC encoder

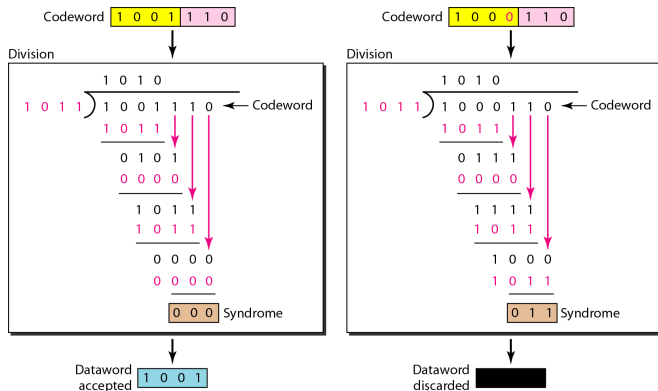
- The dataword is augmented with zeros.
- Division is modulo-2 division, so subtraction is XOR.
- The final remainder becomes the CRC bits.



CRC Division at the Decoder

Figure: Two decoder cases

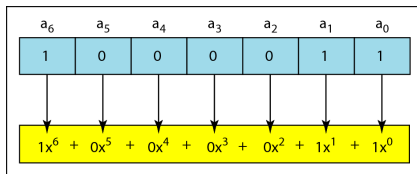
- Left: zero syndrome, so the dataword is accepted.
- Right: nonzero syndrome, so the dataword is discarded.
- A nonzero remainder proves corruption was detected.



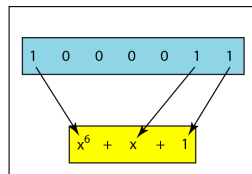
Binary Words as Polynomials

Figure: Polynomial representation

- A bit pattern can be represented by a polynomial.
- Bit positions become powers of x .
- Coefficients are only 0 or 1.



a. Binary pattern and polynomial

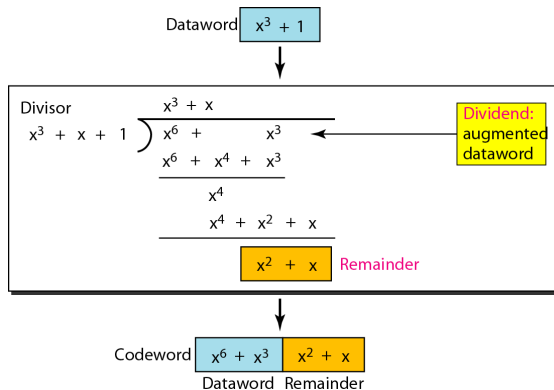


b. Short form

CRC Division Using Polynomials

Figure: Polynomial division

- The dataword polynomial is multiplied by x^r , which appends r zeros.
- Divide by the generator polynomial.
- Append the polynomial remainder to create the codeword.



Generator Polynomial and Syndrome

Generator

The divisor in a cyclic code is normally called the **generator polynomial**, or simply the generator.

Syndrome $s(x)$

If $s(x) \neq 0$, at least one bit is corrupted. If $s(x) = 0$, either no bit is corrupted or an undetected error pattern has occurred.

CRC checking is a polynomial-remainder test.

Cyclic-Code Analysis: What CRC Misses

Undetected pattern

In a cyclic code, error patterns $e(x)$ that are divisible by the generator $g(x)$ are not caught.

$$\frac{C(x) + e(x)}{g(x)}$$

- $C(x)$ is the transmitted codeword polynomial.
- $e(x)$ is the error polynomial.
- If $e(x)$ is also divisible by $g(x)$, the receiver gets zero syndrome even though errors occurred.

Table 10.7: Standard CRC Polynomials

Figure: Common generators

- Different systems standardize different generator polynomials.
- The degree of the polynomial determines the CRC length.
- CRC-32 is commonly associated with LANs.

<i>Name</i>	<i>Polynomial</i>	<i>Application</i>
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Advantages of Cyclic Codes

- They perform well for detecting single-bit errors.
- They perform well for detecting double errors.
- They can detect many odd-number and burst-error patterns.
- They can be implemented efficiently in both hardware and software.
- Hardware implementations are especially fast, which makes CRC attractive for many networks.
- Other cyclic code families use the same cyclic-shift property; CRC is the network-oriented example emphasized here.

Practical reason for CRC

CRC gives strong error detection with simple shift-register or XOR-based implementation.

10.5 Checksum

Definition

A checksum is an error-detection method based on redundancy. It is used by several Internet protocols, though not as the primary data-link-layer check.

Topics in this section

- Basic idea
- One's-complement arithmetic
- Internet checksum

The checksum completes the chapter's survey of error-checking methods.

Checksum Idea

- Like linear and cyclic codes, the checksum adds redundant information.
- A sender computes a value from the data and sends it with the data.
- The receiver recomputes the value and checks whether the result is consistent.
- Checksum is mainly an error-detection method; many systems prefer CRC when stronger detection is needed.

Mental model

The checksum is a compact arithmetic summary of the data. A mismatch suggests the data has been corrupted.

Example 10.18: Sending the Sum

Setup

Suppose the data is five 4-bit numbers: 7, 11, 12, 0, 6.

- The sender sends the data plus the ordinary sum:

(7, 11, 12, 0, 6, 36)

- The receiver adds the first five numbers.
- If the result equals 36, the receiver assumes no error.
- If the result differs, the receiver rejects the data.

Example 10.19: Sending the Checksum

Improved receiver check

Instead of sending the sum, send the negative or complement of the sum.

- The sender sends:

$(7, 11, 12, 0, 6, -36)$

- The receiver adds all received numbers, including the checksum.
- If the result is 0, the receiver assumes no error.
- Otherwise, the receiver detects an error.

One's-Complement Arithmetic

- With n bits, one's-complement arithmetic represents unsigned values from 0 to $2^n - 1$.
- If a sum has more than n bits, the extra leftmost bits are wrapped around and added to the n rightmost bits.
- A negative number is represented by inverting all bits.
- This is equivalent to subtracting the number from $2^n - 1$.

One's-complement checksum arithmetic is addition with end-around carry plus bitwise complement.

Example 10.20: Wrapping 21 into Four Bits

Problem

Represent the number 21 in one's-complement arithmetic using only four bits.

- 21 in binary is 10101, which needs five bits.
- Wrap the leftmost bit around:

$$0101 + 1 = 0110$$

- The four-bit representation is 0110, which is decimal 6.

Checksum Detection: Sender Side

Example 7

Compute an 8-bit checksum over the 16-bit data block 10101001 00111001.

10101001	first 8-bit word
00111001	second 8-bit word
11100010	sum
00011101	checksum

Transmitted bits: 10101001 00111001 00011101.

Checksum Detection: Receiver Side

Example 8

Assume no errors and add the two data words plus the checksum.

10101001	received word 1
00111001	received word 2
00011101	received checksum
11111111	sum
00000000	complement

A complemented sum of zero means the receiver accepts the block.

Checksum Detection: Burst Error Example

Corrupted received data

Suppose a burst error of length 5 changes the received block to 10101111 11111001 00011101.

10101111	corrupted word 1
11111001	corrupted word 2
00011101	checksum
1 11000101	partial sum with carry
00000001	end-around carry
11000110	wrapped sum
00111001	complement

The complement is not zero, so the receiver detects corruption.

Example 10.21: Negative Values

Problem

Represent -6 in one's-complement arithmetic using only four bits.

- Positive 6 is 0110.
- Invert all bits to get the one's-complement negative:

$$0110 \rightarrow 1001$$

- If interpreted as an unsigned value, 1001 is decimal 9.
- Equivalently, $2^4 - 1 - 6 = 15 - 6 = 9$.

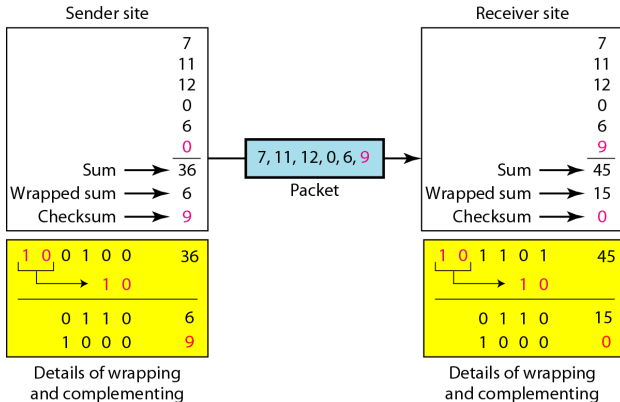
Example 10.22: One's-Complement Checksum

- Start with data values 7, 11, 12, 0, 6 and initial checksum 0.
- The ordinary sum is 36, which cannot be represented in 4 bits.
- Wrap the extra bits to produce wrapped sum 6.
- Complement the wrapped sum: $15 - 6 = 9$, so the sender transmits checksum 9.
- The receiver adds all values including 9, wraps to 15, complements to 0, and accepts the data.

Figure 10.24: Example 10.22

Figure: Sender and receiver arithmetic

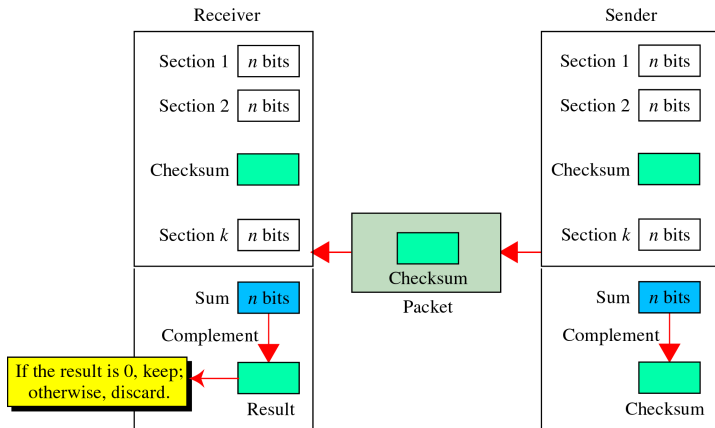
- Sender computes checksum 9.
- Receiver recomputes and obtains 0.
- A zero final checksum means the packet is accepted.



Internet Checksum Generator

Figure: Sender and receiver view

- Sender divides the message into n -bit sections.
- It computes a checksum and sends it with the packet.
- Receiver repeats the addition and accepts only when the final result is zero.



Internet Checksum: Sender Site

1. Divide the message into 16-bit words.
2. Set the checksum word to 0.
3. Add all words, including the checksum word, using one's-complement addition.
4. Complement the sum; the result becomes the checksum.
5. Send the checksum with the data.

The sender creates a checksum that makes the receiver's final complemented sum become zero when no detectable error occurs.

Internet Checksum: Receiver Site

1. Divide the received message, including the checksum, into 16-bit words.
2. Add all words using one's-complement addition.
3. Complement the sum to produce the new checksum.
4. If the checksum value is 0, accept the message.
5. Otherwise, reject the message.

Receiver rule

A zero result is a consistency check, not a mathematical proof that no bit changed.
Some error patterns can still escape a checksum.

Summary (1)

- Data can be corrupted during transmission; some applications require errors to be detected and corrected.
- A single-bit error changes one bit; a burst error affects two or more bits within a span.
- Error-control methods add redundant bits to the data.
- Correction can be done by forward error correction or by detecting the error and retransmitting.
- Many codes use modulo-2 arithmetic, where addition and subtraction both behave like XOR.

Summary (2)

- Hamming distance counts the number of bit positions in which two words differ.
- Minimum Hamming distance determines the detection or correction capability of a block code.
- Cyclic codes are linear block codes whose codewords remain valid under cyclic shifts.
- CRC is a widely used cyclic-code error-detection method based on binary or polynomial division.
- A bit pattern can be represented as a polynomial with coefficients 0 and 1.
- The Internet checksum traditionally uses 16-bit one's-complement arithmetic.

Questions?